Meet dplyr: Making Sales Data Easier to Explore

DSS 665 | R Package Presentation | dplyr | Coffee Shop Sales | Martina Carrazzoni

What is dplyr?

dplyr is an R package that helps us work with data. It makes it easier to select information, create new columns, group records, and summarize results.

Why would we use it?

Imagine a coffee shop with thousands of sales. Instead of checking every row, we can ask simple questions like: Which purchases cost more than $4? Which product category earns the most revenue? Which store sells the most items?

Our example dataset

We will use Coffee Shop Sales from the Maven Analytics Data Playground. It contains 149,116 transaction records, with information such as product category, price, quantity, and store location. The coffee shop chain is fictional, and the dataset is provided for practice.

Five useful functions used with dplyr

- **filter():** selects rows
- **mutate():** creates columns
- **group_by():** organizes rows into groups
- **summarise():** calculates group results
- **arrange():** sorts rows

1. Find the Rows We Need

First, let's practice with **filter()**. This function keeps only the rows that match a condition.

Example 1 | Purchases with a unit price above $4

```
filter(sales, unit_price > 4)
```

Only transaction rows with a unit price greater than $4 will appear.

Example 2 | Coffee purchases only

```
filter(sales, product_category == "Coffee")
```

The output contains only transactions labeled Coffee in the product_category column.

Example 3 | Create a revenue column

Next, mutate() lets us add a calculated column. Revenue for each transaction line is the price of one item multiplied by the number of items sold.

```
mutate(sales, revenue = unit_price * transaction_qty)
```

If a customer buys 2 coffees at $3 each, the revenue for that line is $6.

2. Summarize Sales by Category

Now let's combine functions. **group_by()** organizes the rows into categories, and **summarise()** calculates one result for each category.

Example 4 | Total revenue by product category

```
summarise(
  group_by(
    mutate(sales, revenue = unit_price * transaction_qty),
    product_category
  ),
  total_revenue = sum(revenue)
)
```

This code creates a revenue column by multiplying price by quantity, groups the transactions by product category, and calculates the total revenue for each category.

Example 5 | Average unit price by category

```
summarise(
  group_by(sales, product_category),
  average_price = round(mean(unit_price), 2)
)
```

This code groups transactions by product category and calculates the average unit price for each category. It also rounds the results to two decimal places.

3. Rank Products and Compare Stores

Example 6 | Best-selling products by quantity

We can combine **group_by(), summarise(),** and **arrange()** to rank products. desc() means descending, or highest first.

```
arrange(
  summarise(
    group_by(sales, product_detail),
    units_sold = sum(transaction_qty)
  ),
  desc(units_sold)
)
```

This code groups transactions by product, adds up the total units sold for each product, and sorts the results from highest to lowest to identify the best-selling products.

Example 7 | Units sold at each store

```
summarise(
  group_by(sales, store_location),
  total_units = sum(transaction_qty)
)
```

This code groups transactions by store location and calculates the total number of units sold at each store, allowing us to compare sales quantities across locations.

Key takeaway

dplyr helps turn a large table into answers we can understand. With just a few functions, we filtered purchases, calculated revenue, summarized categories, ranked products, and compared stores.

Dataset: Maven Analytics, Coffee Shop Sales — https://mavenanalytics.io/data-playground/coffee-shop-sales

Package: dplyr — https://dplyr.tidyverse.org/